Experimental Evaluation — Student Build Guide

Telling Application-Layer Volumetric Attacks Apart from Flash-Crowd Events

A step-by-step guide to building the testbed that tests our layered (L1/L2/L3) theory of discriminating information

Target venue: CSF (primary) · DSN, RAID (secondary) · Audience: student researchers building the testbed

How to read this guide

This guide tells you *how to build and run* the experiment correctly. It is written as instructions: do this, capture that, avoid this mistake. The theory behind it lives in a separate paper — here your job is to build an apparatus that lets the theory be tested fairly.

The one idea everything serves

Two kinds of traffic — a **flash crowd** (a legitimate sudden rush of real users) and a **mimicked attack** (a botnet imitating real users) — can look *identical* to any tool that only inspects the traffic. Our claim is that they can still be told apart, but only by adding something that is **not in the traffic**: the exogenous parameter. That parameter is not a single value. It is a **bundle**: the user's **task** (what they are trying to do), the **context** (the human-authored description of what doing that task looks like), and the **agent's / viewer's perspective** (whose goal we judge the traffic against). Your testbed must let a detector use this bundle while proving no traffic-only detector could have reached the same answer.

Central prediction you are testing. A maximally-capable traffic-only learner (M4⋆), given every observable feature, should **fail** to separate a flash crowd from a well-mimicked attack. A semantic detector (M5), given the *same* observations *plus* the exogenous bundle above, should succeed. The gap between them must be explainable only by that bundle — never by extra features.

Contents

1 Topology — what to build **3**

1.1 The four segments and their containers . . . 3

1.1.1 Client segment . . . 3

1.1.2 Edge segment . . . 4

1.1.3 Application segment . . . 4

1.1.4 Monitor segment . . . 4

1.2 The two taps . . . 4

1.3 Ground truth — how to build it and label without bias . . . 4

1.4 Bias traps — read before you run . . . 5

2 The application — what to build and instrument **5**

2.1 The web application . . . 5
2.2 Scenario S1 — Normal (baseline), step by step . . . 6
2.3 Scenario S2 — Flash crowd, step by step . . . 7
2.4 Scenario S3 — Attack, step by step (the ladder) . . . 7
2.5 The three comparisons . . . 7
2.6 Human-like benign browsing — how to implement it . . . 8
2.7 Capturing real user behaviour (the client beacon) . . . 8

3 What to monitor and hold **9**

3.1 L1 — raw data . . . 10
3.2 L2 — everything you can compute from what you observed . . . 10
3.3 L3 — what only the exogenous bundle lets you compute . . . 11
3.4 Attaching labels . . . 11

4 Defenses (detectors) **11**

4.1 Standard tools (first-class L2 baseline) . . . 11
4.1.1 Snort rules (rate + signature) . . . 11
4.1.2 Suricata rules (app-layer rate + a deliberately L0-only signature) . . . 11
4.1.3 Zeek script (stateful rate + source-entropy notice) . . . 12
4.2 M1–M4 — engineered L2 detectors (Python) . . . 13
4.2.1 M1 — volume / rate threshold . . . 13
4.2.2 M2 — per-source concurrency / rate . . . 13
4.2.3 M3 — distributional / Shannon entropy (+ deviation) . . . 13
4.2.4 M4 — g-vulnerability / QIF leakage (the L2 ceiling) . . . 13
4.3 M4$\star$ — the maximal L2 learner (the reviewer-killer) . . . 14
4.4 M5 — the L3 detector (instrumentality, Option A) . . . 14
4.5 The isolation that makes it airtight . . . 14
4.6 What each detector proves . . . 14
4.7 The central figure . . . 15
4.8 The one knob that can quietly rig the result . . . 15

1. Topology — what to build

You should build the testbed with Docker Compose: one container per role. Use **two Docker bridge networks**. The reason is simple: the two networks let you place a capture point (a "tap") on each side — one where traffic enters, one at the server — so you can later match a request to what the server did with it.

Everything is **passive** in this version. Your sensors only *watch* a copy of the traffic. They never sit in the path and never block. This keeps them from disturbing the very traffic you are measuring.

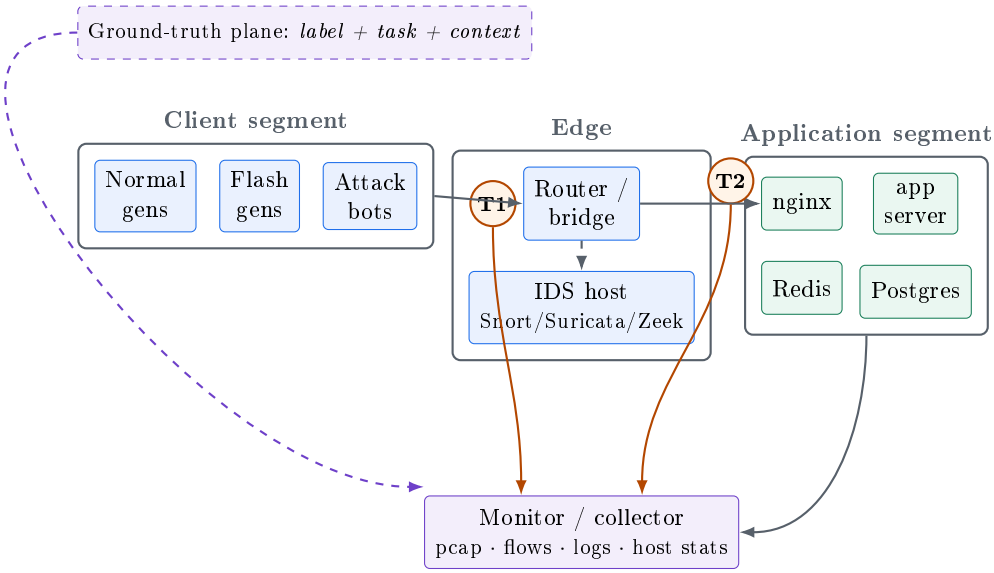


Figure 1: Testbed topology. Solid arrows = traffic; orange circles = passive taps feeding the monitor; purple dashed = the ground-truth bundle, which reaches *only* the monitor and never any detector (§ 1.3).

1.1 The four segments and their containers

Build four groups of containers. Base images are left generic on purpose — pick versions at setup time.

1.1.1 Client segment

Three *profiles*, not three machines. Each profile is a whole population spread across **many source identities**, so that "how many different sources" cannot by itself give the attack away.

- **Normal generators** — several containers. *Image:* a Python image with Locust installed. *Job:* emit many realistic, goal-driven sessions at a calm rate (§ 2.6).
- **Flash generators** — the *same image and the same session code* as Normal, just more of them / spawned faster.
- **Attack bots** — several containers acting as a botnet. *Image:* for the low rungs, a Python image with a simple flood script or flood tool; for the high rungs (L3–L4), a Python image with a **real headless browser** (Selenium or Playwright), because those rungs must reproduce real browser behaviour including the scroll/viewport telemetry (§ 2.7).

1.1.2 Edge segment

- **Router / bridge container** — forwards traffic between the two bridge networks.
- **IDS host** — one container with **Snort**, **Suricata**, and **Zeek** installed. It reads a *mirror* of the edge traffic only. It never blocks.

1.1.3 Application segment

- **nginx** — reverse proxy and cache. *Image:* an nginx image.
- **app server** — *Image:* a Python image running Django or Flask (1–2 replicas). Serves the store in § 2.1.
- **PostgreSQL** — the database. *Image:* a Postgres image.
- **Redis** — the cache. *Image:* a Redis image.

1.1.4 Monitor segment

- **Collector** — one container, out of the traffic path. *Image:* a Linux image with `tcpdump`, `tshark`, Zeek (for flow logs), and Python (pandas) for joining. It receives both taps, the server logs, host/resource stats, and the ground-truth bundle, then writes the joined per-session record.

1.2 The two taps

Tap	Where	Why you need it
T1	At the edge (as traffic enters)	The raw request as it arrives; what the IDS see.
T2	At the application NIC	What actually reached the server, and what the server did (responses, renders, completion).

You need *both* because our semantic layer depends on matching a request (T1) to whether its response was really used (T2). One tap cannot give you both halves.

1.3 Ground truth — how to build it and label without bias

The "ground truth" is the set of facts you know because *you* generated the traffic: which population a session belongs to, and the task/context bundle it was given. This must reach the monitor for scoring, but must **never** reach any detector. That wall is what makes the test honest.

The exogeneity wall

Label, task, and context travel on a **separate channel** straight to the monitor. They are never added to the traffic (not as a header, not as a cookie a detector reads) and never handed to Snort/Suricata/Zeek/M1–M4⋆. If a detector could see the label, the result is contaminated and cannot be fixed afterwards.

How to implement it technically. Give each session a unique `session_id` at generation time. The generator writes one side-record per session to the monitor over a private link (a separate Docker network, or a file/socket the detectors cannot read):

```
session_id, population, task_type, task_target, task_defined_at, context_ref
```

The monitor later joins this to the traffic by `session_id`. The join key across every feed is **5-tuple + session_id + shared timestamp**.

Step-by-step: labelling without bias

1. Assign each population its own set of source identities / subnet. Keep the "source → population" map *only* on the monitor.
2. Stamp every session with a `session_id` and its task/context at the moment it is created — before any traffic is sent (so the task provably predates the traffic).
3. Send the side-record over the private ground-truth channel, never in the HTTP request.
4. Make normal, flash, and attack use the *same base image and network path* — differ only in behaviour, so labels never correlate with a stack fingerprint.
5. On the monitor, join by `session_id`; verify no detector input field contains the label, population, or task.
6. Spot-check: pick random sessions and confirm the label came from the side-channel, not from anything a detector could compute.

1.4 Bias traps — read before you run

Hold all of these, or the result is rigged

1. **Same client stack.** Normal/flash/attack share image and path; differ only in behaviour.
2. **Sensors on a mirror.** IDS watch a copy, never inline.
3. **Symmetric saturation.** Cap the server so flash *and* attack can both overload it the same way; leave headroom for baseline. (See "expensive", § 2.1.)
4. **Shared clock.** One NTP/chrony (or PTP) source for every feed, or the join smears.
5. **Label isolation.** Labels/task/context on a separate channel (§ 1.3).
6. **Matched surges.** Flash and each attack rung match on volume, per-source rate, and source diversity. Only task-instrumentality may differ.
7. **Baseline underneath.** Flash and attack always run *on top of* normal traffic, never against silence.

2. The application — what to build and instrument

2.1 The web application

Build a small but real online store. Prefer adapting an existing demo shop over writing one from scratch, so nobody can say you built it to make the point. It needs several pages, a database behind it, and a cache in front of the heavy page.

Route	What it does	Cost
`/`	Lists the catalogue.	Cheap
`/search?q=...`	Runs a query, returns matches. Part of the normal mix; not the target.	Cheap-ish
`/product/{id}`	**The battleground.** Looks up the product in Postgres, builds personalised recommendations, renders the page.	Expensive
`/cart, /checkout`	State-changing steps; there for realism.	Moderate
static assets	Images/CSS/JS the pages pull in.	Cheap

Data model (minimum). A *products* table (id, name, price, description, category), a *recommendations* relation (product → related products), and an *orders* table (so "completion" is a real, server-verifiable row). Redis caches rendered product fragments; nginx caches static assets.

What makes the product page "dynamic." Each hit does a database lookup, computes recommendations, and renders server-side. That is real work per request — exactly what an attacker wants to trigger.

What "cheap" and "expensive" mean here

A request is **cheap** if the server answers it with almost no work (a static file, or a cached fragment). It is **expensive** if answering costs real resources — a database query, a server-side render, or a large response. The product page is expensive and **asymmetric**: a *tiny* request (`GET /product/42`) forces a *costly* response. Attackers love asymmetry: they spend little, you spend a lot. This definition is used throughout; the symmetric-saturation trap (§ 1.4) and the scenarios (§ 2) rely on it.

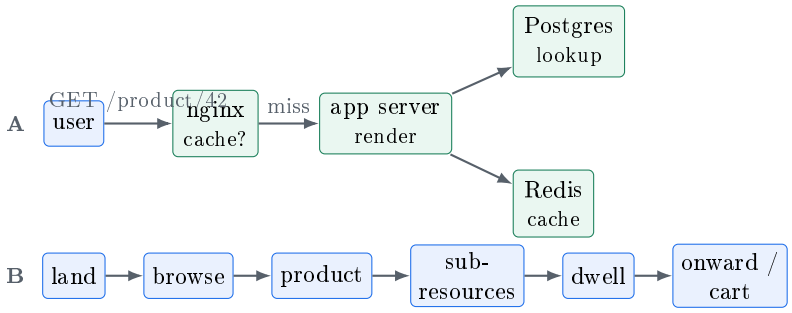


Figure 2: (A) The request path — a cache miss on the product page forces the expensive database lookup + render. (B) A real user's session shape, which the browsing generator (§ 2.6) must reproduce.

2.2 Scenario S1 — Normal (baseline), step by step

1. Start the full application stack; warm the cache with a short low-rate run.
2. Launch the normal generators across their source set at a *calm* arrival rate.
3. Each session follows the shape in Figure 2(B): land → browse a few pages → open a product → fetch its sub-resources → dwell → sometimes go onward/cart → leave. Most sessions bounce; a minority go deep; some abandon.
4. Keep S1 running for the whole experiment — it is the background under S2 and S3.

5. Confirm all feeds are recording (§ 3) and clocks agree.

2.3 Scenario S2 — Flash crowd, step by step

1. Keep S1 running underneath.
2. Pick one product as the "viral" item.
3. Using the *same session code* as normal, spawn a sharp surge of extra real-behaviour sessions, most converging on that one product page.
4. Because these are real sessions, they fetch sub-resources, dwell, and complete at the same rates as normal — only the *volume* is abnormal. The cache gets *hotter* (everyone hits the same key).
5. Record the surge start/stop on the ground-truth channel.

S2 is the false-positive test

A flash crowd is **legitimate**. If a detector flags it, that is a false positive. S2 exists to see which detectors wrongly punish a real crowd.

2.4 Scenario S3 — Attack, step by step (the ladder)

Run the attack as a **ladder of rungs**, each a separate run on top of S1. Keep per-bot rate overlapping flash users and total volume matched to S2.

1. **L0 — naive flood.** Bots re-request the product URL at high rate, uniform timing, fetch no sub-resources, discard the response.
2. **L1 — spread + jitter.** Spread across more sources; add timing randomness so simple rate/entropy rules stop firing.
3. **L2 — page mimicry.** Also fetch the page's sub-resources and hit the *same hot cache key* as the flash crowd.
4. **L3 — behaviour mimicry.** Add human-like dwell, plausible headers, and session structure; run the bots in a real headless browser so the scroll/viewport beacon (§ 2.7) also fires. *At this rung the traffic-only features overlap the flash crowd.*
5. **L4 — genuine completion.** Bots actually consume the response and complete the task. Now only *intent* differs — outside our scope on purpose.

What you should see

Traffic-only detectors separate L0–L1, then **fade to chance by L3**. M5 (which uses the exogenous bundle) holds through L3 and only fades at L4 — *by design*, because L4 removes the last honest difference.

2.5 The three comparisons

- **Normal vs attack** — easy; any decent detector should pass.
- **Flash vs attack** — the hard one. This is the whole paper.
- **Normal vs flash** — must *not* flag. False-positive check.

2.6 Human-like benign browsing — how to implement it

Generate normal *and* flash traffic from a **single Locust session class**, so no tooling fingerprint can separate flash from the high attack rungs. Locust lets you write a user session in Python with realistic waits, page navigation, and sub-resource fetches. The example below shows the *browsing behaviour only* (labelling is handled separately in § 1.3).

```
from locust import HttpUser, task, between
import random

class Shopper(HttpUser):
    wait_time = between(2, 9)          # variable human-like think-time

    def on_start(self):
        self.client.get("/")            # land on the catalogue

    @task
    def browse_and_view(self):
        # browse a few catalogue pages (power-law-ish depth)
        for _ in range(random.choice([1, 1, 2, 2, 3, 5])):
            self.client.get("/", name="/catalogue")

        pid = random.randint(1, 500)
        # open the expensive product page
        self.client.get(f"/product/{pid}", name="/product/[id]")

        # fetch the page's sub-resources, like a real browser
        for asset in ["/static/app.css", "/static/app.js",
                      f"/img/product_{pid}.jpg"]:
            self.client.get(asset, name="/static")

        self.wait()                     # dwell: read before acting

        # sometimes act on what was served (onward click / cart)
        if random.random() < 0.35:
            self.client.get(f"/product/{pid}/recommended", name="/reco")
        if random.random() < 0.15:
            self.client.post("/cart", json={"product": pid}, name="/cart")
```

Turning this into S1 vs S2 vs high-rung S3

Normal (S1): run this class at a low spawn rate over many sources. *Flash (S2):* run the *same* class at a high spawn rate, biased so most pick the one viral `pid`. *High-rung attack (S3, L3–L4):* the *same navigation*, driven through a real headless browser so the scroll/viewport beacon fires too — differing only in whether the task is genuinely served.

2.7 Capturing real user behaviour (the client beacon)

Server logs tell you *when* requests arrived, but not whether a human actually *read* the page. To capture reading behaviour — scroll depth, time on screen — add a small JavaScript beacon to the pages. It reports what the user did back to the app.

```
// injected into product pages; posts lightweight behaviour events
(function () {
  let maxScroll = 0, activeMs = 0, last = Date.now(), visible = true;
```

```
  // how far down the page did the user scroll?
  addEventListener("scroll", () => {
    const d = (scrollY + innerHeight) / document.body.scrollHeight;
    maxScroll = Math.max(maxScroll, d);
  });

  // time the tab was actually visible (real attention)
  document.addEventListener("visibilitychange", () => {
    const now = Date.now();
    if (visible) activeMs += now - last;
    visible = (document.visibilityState === "visible");
    last = now;
  });

  // was the main content actually shown on screen?
  const io = new IntersectionObserver((es) => {
    es.forEach(e => { if (e.isIntersecting) io.seen = true; });
  });
  io.observe(document.querySelector("#product-main"));

  // heartbeat: send accumulated behaviour every few seconds
  setInterval(() => {
    navigator.sendBeacon("/beacon", JSON.stringify({
      sid: window.__sid, maxScroll, activeMs, shown: !!io.seen
    }));
  }, 5000);
})();
```

The app receives these at a `/beacon` endpoint and records them per session.

Important: the beacon is L2, not the semantic signal

Scroll depth, active time, and "content shown" are **observable**. Once emitted, a good enough bot (running a real browser at L3–L4) can reproduce them. So this telemetry is a *richer traffic-only feature* — hand it to M4⋆ like any other observable (§ 3.2). It is **not** the L3 separator; that stays the exogenous task/context bundle. Adding the beacon only makes the "traffic-only tools still fail" result *stronger*.

Two kinds of "dwell" — both are L2

Request-gap dwell: the gap between a session's requests, from server logs/pcap. Always available, no JavaScript. **Reading-dwell**: real on-screen attention, from the beacon above. Richer, but needs the JS. Capture *both*; both are observable, so both go to M4⋆.

3. What to monitor and hold

Guiding rule at this stage

Right now, **do not worry about which layer a measurement belongs to**. Capture everything that might later help prove the point. A missing measurement at run time cannot be recovered; a mis-filed one can be re-sorted in analysis. Over-collect. The *only* thing you must not do is let the ground-truth bundle reach a detector (§ 1.3).

You collect from **five surfaces**, all sharing one clock: (1) network pcap (T1), (2) server pcap (T2),

(3) flows, (4) server & app logs, (5) host/resource telemetry.

3.1 L1 — raw data

From network pcap (T1) and server pcap (T2): packets, bytes, timestamps, 5-tuple, TCP flags, retransmits/drops. Record the *difference* between the two pcaps (what left vs. what arrived). No interpretation yet.

3.2 L2 — everything you can compute from what you observed

Capture all of the following per session and per time window. These feed the traffic-only detectors, including M4⋆; the metrics that *consume* them live in § 4.

Traffic shape (flows + pcap). Requests/s, bytes/s, concurrent connections, per-source rate; inter-arrival mean/variance/higher moments; burstiness; URIs, methods, response codes, request order, session length.

Capture flows at two granularities

Record **flow-level** (Zeek conn.log / IPFIX) *and* keep the **packet-level** pcap. Showing the result at both stops a reviewer claiming it is an artifact of aggregation.

Server & app logs. nginx access/error logs and app logs: response codes, sizes, latencies per request.

Host / resource telemetry (the fifth surface). CPU, memory, thread/worker counts, queue depths — read *inside* the app/server containers (cAdvisor, node-exporter, or `/proc`), not from the taps.

Expected to be useless on the hard axis — capture anyway

Because flash and attack are tuned to overload the server the *same way* (§ 1.4), resource metrics should **not** separate them. That is the point: they become some of the flattest lines in the co-movement figure (§ 4.7), showing "the server is dying" says nothing about *which* surge is killing it.

Consumption-as-observed. Sub-resource fetches; request-gap dwell; reading-dwell/scroll/viewport from the beacon (§ 2.7); next-request target.

Cache-hotness (first-class feature). Capture, per window:

- hit-ratio time series — Redis `INFO stats` (`keyspace_hits`/`keyspace_misses`);
- per-key hotness — Redis `--hotkeys` / `OBJECT FREQ` (LFU);
- per-request cache outcome — nginx `$upstream_cache_status` (HIT/MISS/EXPIRED);
- unique-key cardinality per window.

Cache-hotness separates the low rungs (cache-busting attack vs. a hot flash crowd) and *converges* once a smart attack targets the hot key — the same fade-with-rung pattern as everything else.

Raw distributions for the metrics. Also store the *count tables* the entropy and g-leakage metrics are computed from: the source-IP distribution, request-type distribution, and inter-arrival bins per window. These are the *inputs* to M3/M4 (§ 4); the metric computations stay in Defenses so "what

we observed" and "what we did with it" stay separate.

3.3 L3 — what only the exogenous bundle lets you compute

From server logs joined to the ground-truth plane, per session: the **task** label; the **context** (human-authored required-consumption spec); **response uptake** (was the body really consumed vs. discarded); **content-coupling** (does the next request reference content only present in *this* session's served response?); **completion** (server-verified); and the resulting **instrumentality score** (§ 4.4).

3.4 Attaching labels

The full labelling mechanism and the bias-free, step-by-step procedure live with the ground-truth build in § 1.3. In short: labels come from the side-channel, are joined by `session_id` at the monitor, and never touch a detector.

4. Defenses (detectors)

Order the detectors by the layer they work on. Everything is passive and reads the feeds from § 3.

Build rule: sensors extract, Python decides

Let Zeek and the other sensors *extract* features. Compute *all* detector logic in one Python pipeline over the saved feeds. This keeps the feature quantisation identical across M1–M4⋆ (§ 4.8), lets M4/M4⋆ host maths Zeek cannot, and makes the whole thing one reproducible artifact.

4.1 Standard tools (first-class L2 baseline)

Snort and Suricata do signatures + rate/threshold rules. Zeek is the strongest traffic-only tool (stateful, scriptable) — so it is the "even Zeek cannot cross" exhibit. These fire their *own* native alerts as a real-world baseline, kept separate from the Python M1–M3.

4.1.1 Snort rules (rate + signature)

```
# Per-source rate to the product page
alert tcp any any -> $VICTIM 80 ( \
  msg:"M-SNORT rate to product page"; flow:to_server,established; \
  content:"GET"; http_method; content:"/product/"; http_uri; \
  detection_filter:track by_src, count 100, seconds 10; \
  sid:1000001; rev:1; )

# Aggregate flood to the endpoint (many sources)
alert tcp any any -> $VICTIM 80 ( \
  msg:"M-SNORT aggregate flood"; flow:to_server,established; \
  content:"/product/"; http_uri; \
  detection_filter:track by_dst, count 5000, seconds 10; \
  sid:1000002; rev:1; )
```

4.1.2 Suricata rules (app-layer rate + a deliberately L0-only signature)

```
alert http $EXTERNAL_NET any -> $HOME_NET any ( \
```

```
  msg:"M-SURI per-source rate to product page"; flow:to_server,established; \
  http.method; content:"GET"; http.uri; content:"/product/"; \
  detection_filter:track by_src, count 100, seconds 10; \
  sid:2000001; rev:1; )

# Naive-flood tell: known flood-tool User-Agent (catches L0 only)
alert http $EXTERNAL_NET any -> $HOME_NET any ( \
  msg:"M-SURI suspicious flood UA"; flow:to_server; \
  http.user_agent; pcre:"/^(hulk|goldeneye|python-requests)/i"; \
  sid:2000002; rev:1; )

# Aggregate volume to endpoint
alert http $EXTERNAL_NET any -> $HOME_NET any ( \
  msg:"M-SURI aggregate volume"; flow:to_server; \
  http.uri; content:"/product/"; \
  detection_filter:track by_dst, count 5000, seconds 10; \
  sid:2000003; rev:1; )
```

The User-Agent rule is deliberately L0-only: it lets you *show* signature detection dies the moment the bot sets a normal UA.

4.1.3 Zeek script (stateful rate + source-entropy notice)

```
@load base/protocols/http
redef enum Notice::Type += { High_Req_Rate, Low_Source_Entropy };

global req_count: table[addr] of count &default=0 &create_expire=10sec;
global window_hits: table[addr] of count &default=0;

event http_request(c: connection, method: string, orig: string,
                   unescaped: string, version: string) {
  if ( /^\/product\// in orig ) {
    ++req_count[c$id$orig_h];
    ++window_hits[c$id$orig_h];
    if ( req_count[c$id$orig_h] == 100 )
      NOTICE([$note=High_Req_Rate, $src=c$id$orig_h,
              $msg="source exceeded 100 product-page reqs/10s"]);
  }
}

# Each window: Shannon entropy of the source-IP request distribution
event entropy_check() {
  local total = 0.0;
  for ( a in window_hits ) total += window_hits[a];
  local H = 0.0;
  for ( a in window_hits ) {
    local p = window_hits[a] / total;
    if ( p > 0 ) H += -p * log10(p)/log10(2);
  }
  if ( H < entropy_floor )
    NOTICE([$note=Low_Source_Entropy, $msg=fmt("source entropy %.3f", H)]);
  clear_table(window_hits);
  schedule 10sec { entropy_check() };
}
```

4.2 M1–M4 — engineered L2 detectors (Python)

Per window `w` (e.g. 10 s). Group requests by source `i` with counts `c_i`, total `N`, over `S` sources; `p_i = c_i / N`.

4.2.1 M1 — volume / rate threshold

```
def M1(window_count, baseline_mean, baseline_std, k=3):
    rate = window_count / WINDOW_SECONDS
    return rate > baseline_mean + k * baseline_std   # attack-surge flag
```

Separates surge from calm; blind to flash-vs-attack from L0.

4.2.2 M2 — per-source concurrency / rate

```
import numpy as np
def M2(counts_by_source, per_source_ceiling):
    rates = np.array(list(counts_by_source.values())) / WINDOW_SECONDS
    return rates.max() > per_source_ceiling, np.percentile(rates, 95)
```

Dies once per-bot rate overlaps flash users.

4.2.3 M3 — distributional / Shannon entropy (+ deviation)

```
def shannon_entropy(counts):
    c = np.array(list(counts.values()), dtype=float)
    p = c / c.sum(); p = p[p > 0]
    return float(-(p * np.log2(p)).sum())            # bits

def M3(counts_by_source, baseline_H, ewma_std, k=3):
    H = shannon_entropy(counts_by_source)
    deviation = abs(H - baseline_H)
    return deviation > k * ewma_std, H               # flag, entropy
```

Request-type entropy and cache-hit-ratio use the *same* formula over different count tables — feed all three. Separates low rungs; converges as the attack shapes its distributions and hits the hot key.

4.2.4 M4 — g-vulnerability / QIF leakage (the L2 ceiling)

For a secret label `y in {flash, attack}` and observed feature `o`: Bayes vulnerability `V(Y|O) = Sum_o P(o) max_y P(y|o)`; min-entropy leakage `L = log2( V(Y|O)/V(Y) )`, `V(Y)=max_y P(y)`. Leakage $\to 0$ means the traffic carries *no* information separating the labels.

```
from collections import defaultdict
def M4_gvuln_leakage(observations, labels):
    N = len(labels)
    prior = defaultdict(int)
    for y in labels: prior[y] += 1
    V_prior = max(prior.values()) / N

    joint = defaultdict(lambda: defaultdict(int))
    obs_count = defaultdict(int)
    for o, y in zip(observations, labels):
        joint[o][y] += 1; obs_count[o] += 1
    V_post = sum(max(joint[o].values()) for o in obs_count) / N
```

```
    leakage = np.log2(V_post / V_prior)              # bits; ~0 => L2 empty
    return V_post, leakage
```

This leakage is your convergence metric: positive at L0–L1, collapsing toward 0 by L3.

4.3 M4⋆ — the maximal L2 learner (the reviewer-killer)

Train a strong model (gradient boosting / deep sequence model) on the *entire* L2 feature set from § 3.2 — structure, statistics, cache-hotness, *and* all observable consumption including the beacon. Let it extract anything computable from the traffic.

Why M4⋆ matters

M4⋆ gets *everything* M5 gets **except** the task/context bundle. If M4⋆, with all features and a strong learner, still **plateaus** at the top rung, then "you just added features" is refuted *in your own paper*: you gave the traffic-only side everything and it still could not cross.

4.4 M5 — the L3 detector (instrumentality, Option A)

M5 reads the joined record *plus* the human-authored task/context (§ 3.3). It scores, per session, the fraction of the task's required-consumption the session actually performed, then thresholds it. It is the *only* detector holding the exogenous bundle. It holds through L3 and fades at L4 by design. (A likelihood-ratio form, M5-B, mirroring M4's maths, is planned for later; the context is authored to support both a checklist reading and a distributional one.)

4.5 The isolation that makes it airtight

Same eyes, one extra input

M4⋆ and M5 see the **same observations**. The only difference is M5's task/context bundle. So any gap between them at the top rung is caused by that bundle — *never* by extra features. This is the experimental heart of the claim.

4.6 What each detector proves

Detector(s)	What it establishes
Snort / Suricata / Zeek + M1–M3	The *practical* failure — real, well-configured tools miss the mimicry.
M4	The *theoretical* L2 ceiling — the best hand-built statistic fails (leakage $\to 0$).
M4⋆	The *empirical* refutation of "just add features" — best learner, all features, still plateaus.
M5	The *resolution* — only the exogenous task/context bundle separates them.

4.7 The central figure

Sweep every traffic-only detector's operating point (rate, source entropy, inter-arrival deviation, sub-resource stats, cache-hotness, resource load — all of it). Show that **flash crowd and top-rung attack move together on every one**: there is no traffic-only setting that lets the flash crowd through while stopping the attack. Then M5, at a single operating point using the bundle, separates them. The message: *no traffic-only detector can exist that does this — and here is the layer where one can.*

4.8 The one knob that can quietly rig the result

Quantisation fairness

M4 and M4⋆ need continuous features binned into `o`. Bin too coarsely and you understate the traffic-only ceiling (unfair to the baseline); too finely and you overfit and overstate it. Fix the binning by a stated rule (e.g. equal-frequency bins fit on baseline only), report a sensitivity analysis, and use the **identical** binning across M1–M4⋆. Running all detectors in one Python pipeline over the same feed (§ 4) is what guarantees this.

Open items carried forward (not in this guide)

- **Context-encoding scheme** — how the human writes the context per task type, supporting both a checklist and a distributional reading.
- **M5 (A) scoring detail** — the concrete instrumentality score and threshold.
- **Per-session join schema** — the exact record the monitor writes.
- **Population sizing / rate-matching calibration** — exact counts, rates, worker counts, durations.
- **Heterogeneous second experiment** — another domain to show the paradigm generalises.
- **The top-rung-gap belief** — the argument that observable consumption is reproducible without a task, so M4⋆ must plateau (to be pressure-tested).